

01 — Exploratory Data Analysis

ADHD EEG Classification Project

Dataset: IEEE 19-Channel EEG — 61 ADHD, 60 Healthy Children (ages 7–12)

Sampling Rate: 128 Hz | Format: single CSV, one row per EEG sample

Goals:

- Understand data shape, class balance, and subject recording lengths
- Visualize raw EEG signals for ADHD vs. Control
- Compute Power Spectral Density (PSD) using Welch's method
- Compare band power (Delta, Theta, Alpha, Beta) and the theta/beta ratio across groups

0. Imports & Config

```
In [10]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import signal
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='darkgrid', palette='muted')
plt.rcParams['figure.dpi'] = 120
plt.rcParams['font.size'] = 11
os.makedirs('plots', exist_ok=True)

SAMPLING_RATE = 128 # Hz

BANDS = {
    'Delta': (1, 4),
```

```

    'Theta': (4, 8),
    'Alpha': (8, 12),
    'Beta': (12, 30)
}

CHANNELS = ['Fp1', 'Fp2', 'F3', 'F4', 'C3', 'C4', 'P3', 'P4', 'O1', 'O2',
            'F7', 'F8', 'T7', 'T8', 'P7', 'P8', 'Fz', 'Cz', 'Pz']

COLORS = {'ADHD': '#e07b7b', 'Control': '#7bb8e0'}

print('Imports OK')

```

Imports OK

1. Load Data

```

In [2]: FILE_PATH = r'C:\Users\albader\OneDrive\Desktop\Data Science and Analytics\Projects\adhd-eeg-classification\data\adhddata.csv'

df_raw = pd.read_csv(FILE_PATH)

adhd_ids = df_raw[df_raw['Class'] == 'ADHD']['ID'].unique()
control_ids = df_raw[df_raw['Class'] == 'Control']['ID'].unique()

def get_subject(subject_id):
    """Return (ndarray of shape (n_samples, 19), label string) for a subject."""
    rows = df_raw[df_raw['ID'] == subject_id]
    return rows[CHANNELS].values.astype(float), rows['Class'].iloc[0]

print(f'Total samples : {len(df_raw):,}')
print(f'Subjects      : {df_raw["ID"].nunique()} (ADHD: {len(adhd_ids)}, Control: {len(control_ids)})')
print(f'Samples/class : ADHD {df_raw["Class"].value_counts()["ADHD"]:,} | Control {df_raw["Class"].value_counts()["Control"]:,}')

```

```

Total samples : 2,166,383
Subjects      : 121 (ADHD: 61, Control: 60)
Samples/class : ADHD 1,207,069 | Control 959,314

```

2. Class Balance & Recording Lengths

```
In [3]: lengths = []
        for sid in df_raw['ID'].unique():
            rows = df_raw[df_raw['ID'] == sid]
            label = rows['Class'].iloc[0]
            lengths.append({'ID': sid, 'Class': label,
                           'duration_s': len(rows) / SAMPLING_RATE})
        df_lengths = pd.DataFrame(lengths)

        print(df_lengths.groupby('Class')['duration_s']
              .agg(['mean', 'min', 'max']).round(1))
```

	mean	min	max
Class			
ADHD	154.6	72.5	337.9
Control	124.9	62.4	203.0

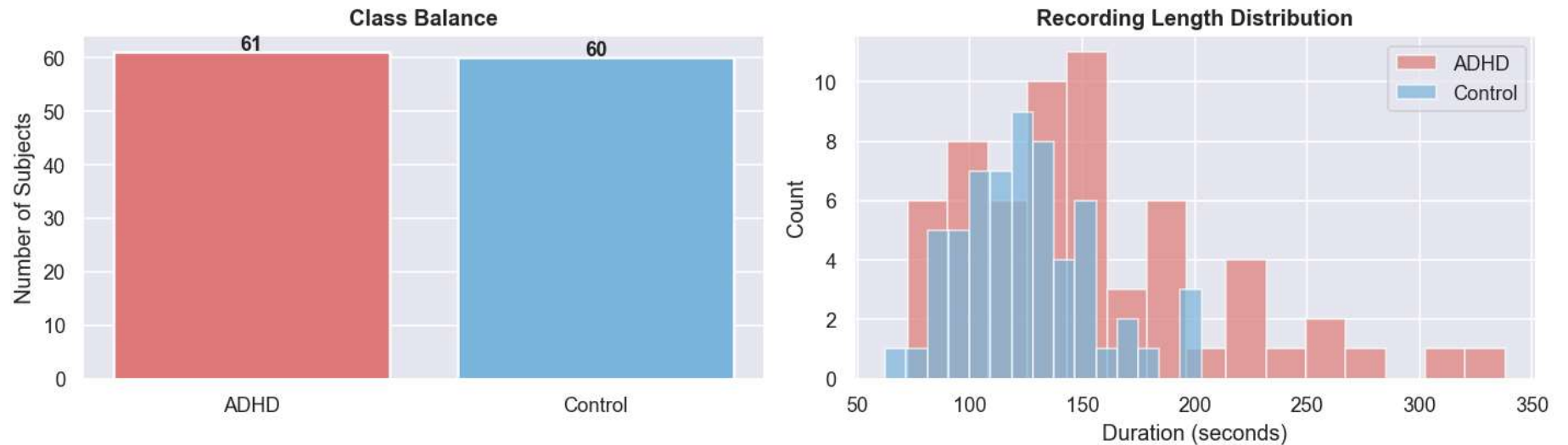
```
In [4]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))

        # Class balance bar chart
        axes[0].bar(['ADHD', 'Control'], [len(adhd_ids), len(control_ids)],
                    color=[COLORS['ADHD'], COLORS['Control']], edgecolor='white', linewidth=1.5)
        axes[0].set_title('Class Balance', fontweight='bold')
        axes[0].set_ylabel('Number of Subjects')
        for i, v in enumerate([len(adhd_ids), len(control_ids)]):
            axes[0].text(i, v + 0.3, str(v), ha='center', fontweight='bold')

        # Recording Length distribution
        for label, grp in df_lengths.groupby('Class'):
            axes[1].hist(grp['duration_s'], bins=15, alpha=0.7,
                        color=COLORS[label], label=label, edgecolor='white')
        axes[1].set_title('Recording Length Distribution', fontweight='bold')
        axes[1].set_xlabel('Duration (seconds)')
        axes[1].set_ylabel('Count')
        axes[1].legend()

        plt.suptitle('Dataset Overview', fontsize=14, fontweight='bold')
        plt.tight_layout()
        plt.savefig('plots/01_dataset_overview.png', bbox_inches='tight')
        plt.show()
```

Dataset Overview



3. Raw EEG Signal Visualization

5 seconds of raw signal for one ADHD subject vs. one Control subject.

```
In [5]: def plot_raw_eeg(data, label, duration_sec=5, n_ch=8):
    n_samples = duration_sec * SAMPLING_RATE
    segment = data[:n_samples, :n_ch]
    t = np.arange(n_samples) / SAMPLING_RATE

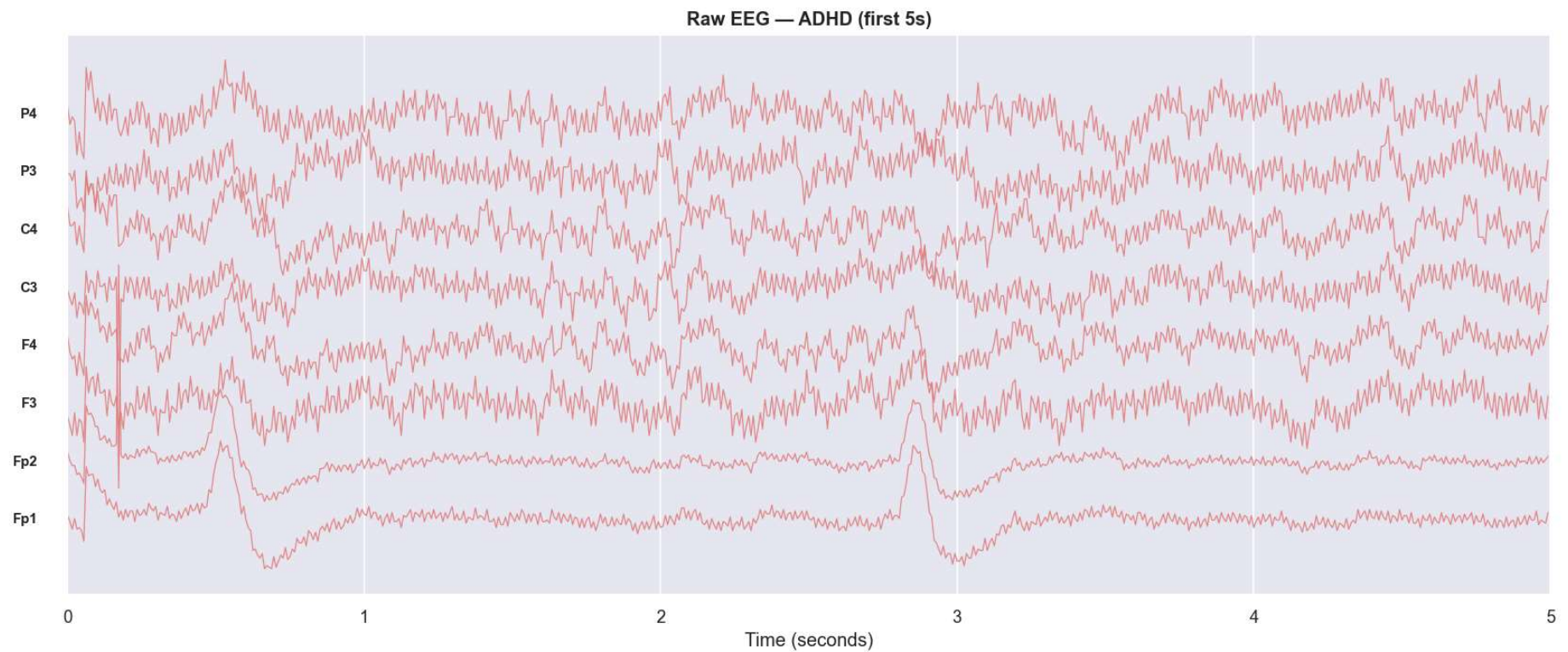
    fig, ax = plt.subplots(figsize=(14, 6))
    for i in range(n_ch):
        ch = segment[:, i]
        ch_norm = (ch - ch.mean()) / (ch.std() + 1e-8)
        ax.plot(t, ch_norm + i * 3.5, color=COLORS[label], alpha=0.85, linewidth=0.8)
        ax.text(-0.1, i * 3.5, CHANNELS[i], ha='right', va='center',
                fontsize=9, fontweight='bold')
    ax.set_xlabel('Time (seconds)')
    ax.set_title(f'Raw EEG - {label} (first {duration_sec}s)', fontweight='bold')
    ax.set_yticks([])
    ax.set_xlim(0, duration_sec)
```

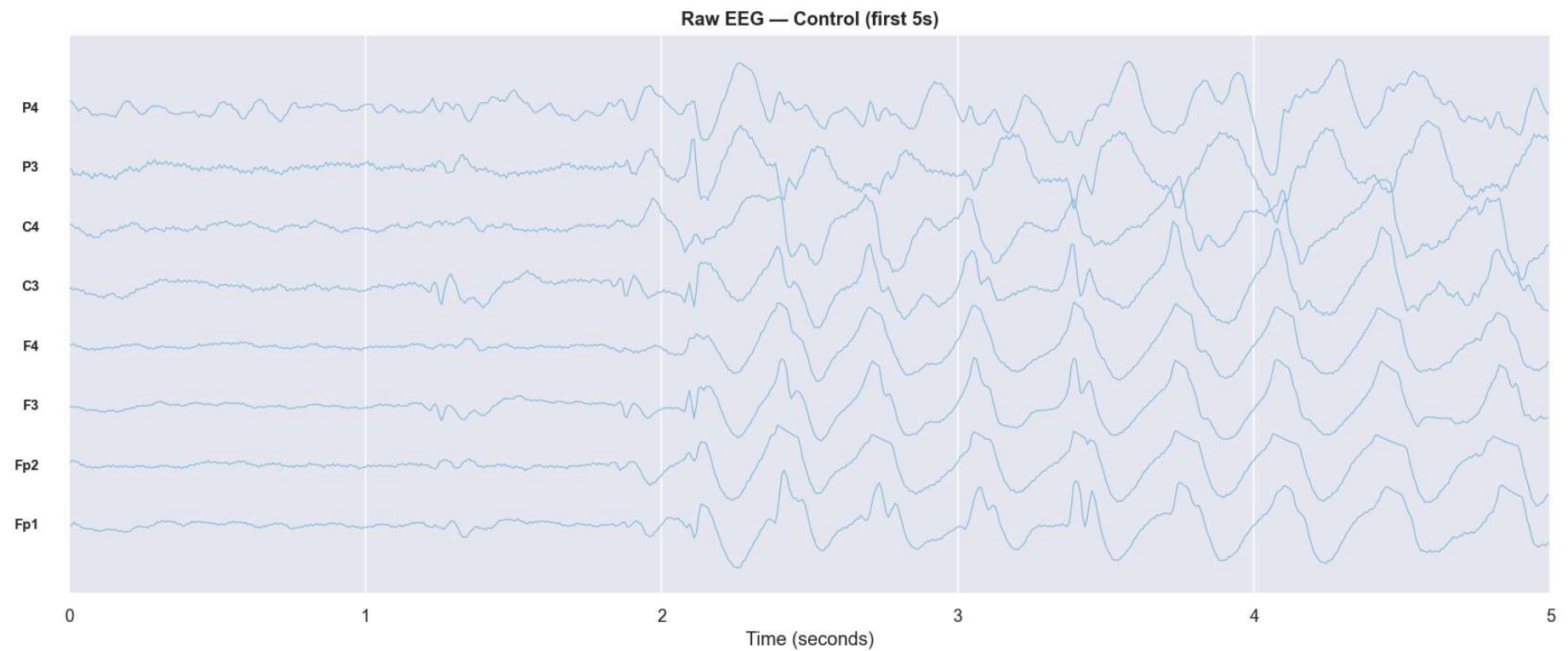
```
plt.tight_layout()
return fig

adhd_data, _ = get_subject(adhd_ids[0])
control_data, _ = get_subject(control_ids[0])

fig = plot_raw_eeg(adhd_data, 'ADHD')
fig.savefig('plots/02_raw_eeg_adhd.png', bbox_inches='tight')
plt.show()

fig = plot_raw_eeg(control_data, 'Control')
fig.savefig('plots/02_raw_eeg_control.png', bbox_inches='tight')
plt.show()
```





4. Power Spectral Density (Welch's Method)

Welch's method averages overlapping FFT windows — more stable than a raw FFT on noisy EEG.

```
In [6]: def compute_psd(data, fs=SAMPLING_RATE, nperseg=256):
        """Average PSD across all 19 channels."""
        psds = []
        for ch in range(data.shape[1]):
            freqs, psd = signal.welch(data[:, ch], fs=fs, nperseg=nperseg)
            psds.append(psd)
        return freqs, np.mean(psds, axis=0)

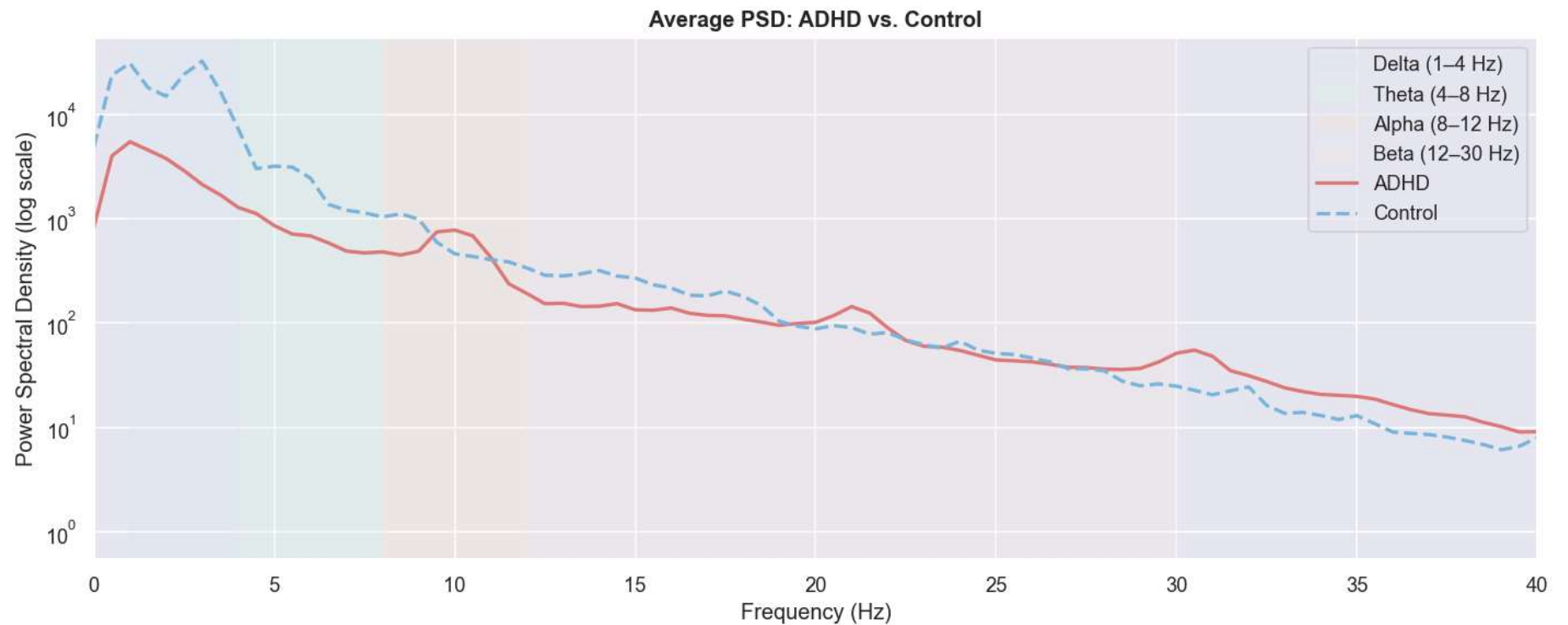
        freqs_a, psd_a = compute_psd(adhd_data)
        freqs_c, psd_c = compute_psd(control_data)
```

```

band_colors = {'Delta': '#d4e6f1', 'Theta': '#d5f5e3', 'Alpha': '#fdebd0', 'Beta': '#f9ebeb'}

fig, ax = plt.subplots(figsize=(12, 5))
for band, (lo, hi) in BANDS.items():
    ax.axvspan(lo, hi, alpha=0.25, color=band_colors[band], label=f'{band} ({lo}-{hi} Hz)')
ax.semilogy(freqs_a, psd_a, color=COLORS['ADHD'], linewidth=2, label='ADHD')
ax.semilogy(freqs_c, psd_c, color=COLORS['Control'], linewidth=2, label='Control', linestyle='--')
ax.set_xlim(0, 40)
ax.set_xlabel('Frequency (Hz)')
ax.set_ylabel('Power Spectral Density (log scale)')
ax.set_title('Average PSD: ADHD vs. Control', fontweight='bold')
ax.legend(loc='upper right')
plt.tight_layout()
plt.savefig('plots/03_psd_comparison.png', bbox_inches='tight')
plt.show()

```



5. Band Power Across All 121 Subjects

Compute mean band power per subject, then compare groups.

The **theta/beta ratio** is a well-established ADHD biomarker — elevated in ADHD children.

```
In [7]: def band_power(data, fs=SAMPLING_RATE, nperseg=256):
        """Return absolute power in each band, averaged across channels."""
        freqs, psd_mean = compute_psd(data, fs, nperseg)
        freq_res = freqs[1] - freqs[0]
        return {band: np.sum(psd_mean[(freqs >= lo) & (freqs <= hi)]) * freq_res
                for band, (lo, hi) in BANDS.items()}

records = []
all_ids = df_raw['ID'].unique()

for i, sid in enumerate(all_ids):
    data, label = get_subject(sid)
    bp = band_power(data)
    bp['label'] = label
    bp['subject'] = sid
    records.append(bp)
    if (i + 1) % 20 == 0:
        print(f' Processed {i+1}/{len(all_ids)} subjects...')

df_bp = pd.DataFrame(records)
df_bp['Theta_Beta_Ratio'] = df_bp['Theta'] / df_bp['Beta']

print(f'\nDone. Shape: {df_bp.shape}')
df_bp.groupby('label')[['Delta', 'Theta', 'Alpha', 'Beta', 'Theta_Beta_Ratio']].mean().round(4)

Processed 20/121 subjects...
Processed 40/121 subjects...
Processed 60/121 subjects...
Processed 80/121 subjects...
Processed 100/121 subjects...
Processed 120/121 subjects...
```

Done. Shape: (121, 7)

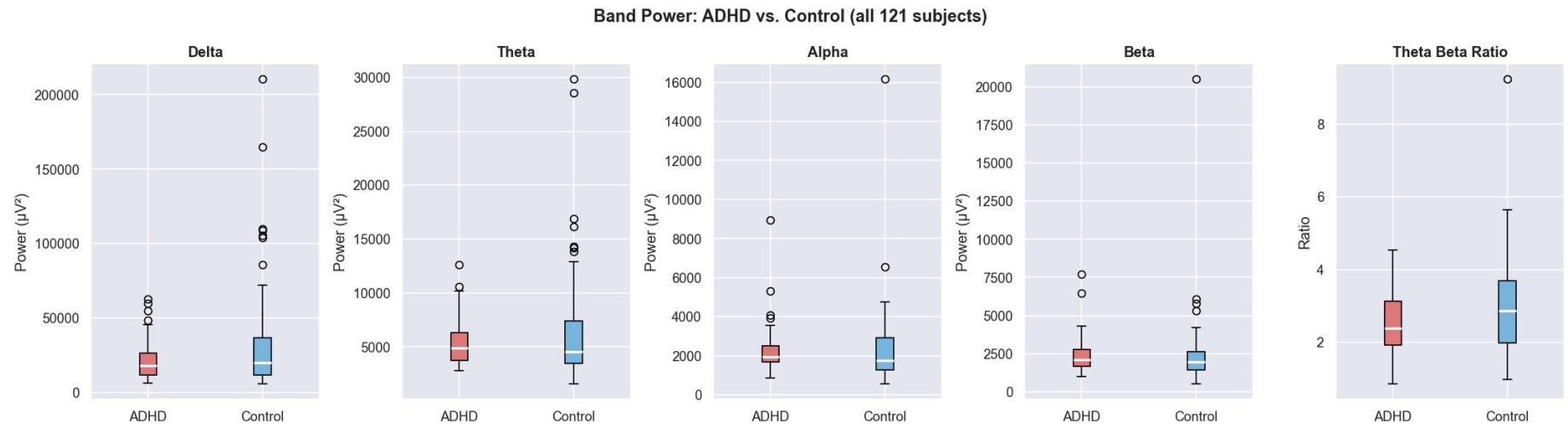
Out[7]:

	Delta	Theta	Alpha	Beta	Theta_Beta_Ratio
label					
ADHD	21061.9111	5454.4216	2257.6187	2334.5932	2.5435
Control	34257.8692	6690.2033	2360.3908	2457.5156	2.9992

```
In [11]: features = ['Delta', 'Theta', 'Alpha', 'Beta', 'Theta_Beta_Ratio']

fig, axes = plt.subplots(1, 5, figsize=(18, 5))
for ax, feat in zip(axes, features):
    data_a = df_bp[df_bp.label == 'ADHD'][feat].values
    data_c = df_bp[df_bp.label == 'Control'][feat].values
    bp = ax.boxplot([data_a, data_c], patch_artist=True,
                    labels=['ADHD', 'Control'],
                    medianprops=dict(color='white', linewidth=2))
    bp['boxes'][0].set_facecolor(COLORS['ADHD'])
    bp['boxes'][1].set_facecolor(COLORS['Control'])
    ax.set_title(feat.replace('_', ' '), fontweight='bold')
    ax.set_ylabel('Power ( $\mu V^2$ )' if feat != 'Theta_Beta_Ratio' else 'Ratio')

plt.suptitle('Band Power: ADHD vs. Control (all 121 subjects)', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('plots/04_band_power_boxplots.png', bbox_inches='tight')
plt.show()
```



6. EDA Summary

```
In [9]: print('='*45)
print('EDA SUMMARY')
print('='*45)
print(f'Total subjects : {df_raw["ID"].unique()}')
print(f'  ADHD          : {len(adhd_ids)}')
print(f'  Control         : {len(control_ids)}')
print(f'Total samples   : {len(df_raw):,}')
print(f'Sampling rate    : {SAMPLING_RATE} Hz')
print(f'Channels         : {len(CHANNELS)}')
print()
print('Mean recording duration (seconds):')
print(df_lengths.groupby('Class')['duration_s'].mean().round(1).to_string())
print()
print('Mean Theta/Beta Ratio:')
print(df_bp.groupby('label')['Theta_Beta_Ratio'].mean().round(4).to_string())
print()
print('Plots saved to: plots/')
```

```
=====
EDA SUMMARY
=====
Total subjects : 121
  ADHD         : 61
  Control      : 60
Total samples  : 2,166,383
Sampling rate  : 128 Hz
Channels       : 19
```

Mean recording duration (seconds):

Class

ADHD 154.6

Control 124.9

Mean Theta/Beta Ratio:

label

ADHD 2.5435

Control 2.9992

Plots saved to: plots/

Next: `02_feature_extraction.ipynb` — Extract per-channel band power, theta/beta ratio, and statistical features into a model-ready feature matrix (one row per subject).